1. **Q-2**

Textual MEDIUM 5 marks

Build the 16-bit Register chip in HDL. Show read/write operations for three sample inputs and capture simulation output.

YOUR ANSWER

CHIP Register { IN in[16], load ; OUT out[16]; PARTS: Bit(in=in[0], load=load, out=out[0]); Bit(in=in[1],load=load , out=out[1]); Bit(in=in[2],load=load, out=out[2]); Bit(in=in[3],load=load,out=out[3]); . . . . . . Bit(in=in[15],load=load,out=out[15]); operation input load output write 1 0000000000001010 1 0000000000001010 read/hold 1111111111110000 0 0000000000001010 write 2 0000000011111111 1 0000000011111111

Implement XOR, Mux (2-way, 4-way), and DMux chips in HDL. Show truth tables and verify against the Nand2Tetris test scripts.

YOUR ANSWER

XOR hdl CHIP Xor { IN a , b; OUT out; PARTS: Or(a=a, b=b, out=orAB); And(a=a, b=b, out=andAB); Not (in=andAB, out=notAnd); And(a=orAB, b=notAnd, out=out); } MUX-2WAY CHIP Mux { IN a, b, sel ; OUT out ; PARTS: Not(in=sel, out=notSel); And(a=a, b=notSel, out=andA); And(a=b, b=sel, out=andB); Or (a=andA, b=andB, out=out); } MUX-4WAY CHIP Mux4Way16 { IN a, b, c, d, sel[2]; OUT out; PARTS: Mux(a=a, b=b, sel=sel[0] , out=ab); Mux(a=c, b=d, sel=sel[0], out=cd); Mux(a=ab, b=cd, sel=sel[1], out=out); } D-MUX CHIP DMux { IN in, sel; OUT a, b; PARTS: Not(in=sel, out=notSel); And(a=in, b=notSel, out=a); And(a=in, b=sel, out=b); } TRUTH -TABLE XOR a b out 0 0 0 0 1 1 1 0 1 1 1 0 MUX sel out 0 a 1 b DMUX sel a b 0 in 0 1 0 in sel[1] sel[0] output 0 0 a 0 1 b 1 0 c 1 1 d

1. **Q-3**

Textual MEDIUM 5 marks

Build the Half Adder and Full Adder chips in HDL. Trace the carry propagation for a sample input and show simulation screenshots.

YOUR ANSWER

CHIP HalfAdder{ IN a, b; //1-bit inputs OUT sum, //Right bit of a + b carry; //Left bit of a + b PARTS: Xor(a=a , b=b ,out= sum); And(a=a, b=b, out= carry); } CHIP FullAdder{ IN a,b,c; //1-bit inputs OUT sum, //Right bit of a + b + c PARTS: HalfAdder(a=a, b=c, sum=s1 , carry=c1); HalfAdder(a=s1 , b=c, sum=sum , carry=c2); or(a=c1 , b=c2 , out=carry); }

Q 4

Build the 16-bit Ripple-Carry Adder in HDL. Test with two sample 16-bit inputs. Explain overflow behavior and two's complement representation.

YOUR ANSWER

CHIP Add16 { IN a[16], b[16]; OUT out[16]; PARTS: halfadder(a=a[0], b=b[0],sum=out[0],carry=c1);
FullAdder(a=a[1],b=b[1],c=c1,sum=out[1],carry=c2); FullAdder(a=a[2],b=b[2],c=c2,sum=out[2],carry=c3);
FullAdder(a=a[3],b=b[3],c=c3,sum=out[3],carry=c4); FullAdder(a=a[4],b=b[4],c=c4,sum=out[4],carry=c5);
FullAdder(a=a[5],b=b[5],c=c5,sum=out[5],carry=c6); FullAdder(a=a[6],b=b[6],c=c6,sum=out[6],carry=c7);
FullAdder(a=a[7],b=b[7],c=c7,sum=out[7],carry=c8); FullAdder(a=a[8],b=b[8],c=c8,sum=out[8],carry=c9);
FullAdder(a=a[9],b=b[9],c=c9,sum=out[9],carry=c10); FullAdder(a=a[10],b=b[10],c=c10,sum=out[10],carry=c11);
FullAdder(a=a[11],b=b[11],c=c11,sum=out[11],carry=c12); FullAdder(a=a[12],b=b[12],c=c12,sum=out[12],carry=c13);
FullAdder(a=a[13],b=b[13],c=c13,sum=out[13],carry=c14); FullAdder(a=a[14],b=b[14],c=c14,sum=out[14],carry=c15);
FullAdder(a=a[15],b=b[15],c=c15,sum=out[15],carry=c16); } TWO SAMPLE 16 BIT: sample 1 a= 0000 0000 0000
0101 b= 0000 0000 0000 1010 out = 0000 0000 0000 1111 5 + 10 = 15 <ol start="2"> sample 2 a = 0000 0000
0001 1001 b = 0000 0000 0001 1110 out = 0000 0000 0011 0111 25 + 30 = 55

TWO'S COMPLEMENT REPRESENTATION:
represent -5 in 16 bits: +5 = 0000 0000 0000 0101 Invert = 1111 1111 1111 1010 Add 1 = 1111 1111 1111 1011 -5 = 1111 1111 1111
1011 <ol start="2"> represent -55 in 16 bits: +55= 0000 0000 0011 1001 Invert = 1111 1111 1100 0110 Add1 = 1111 1111
1100 0111 -55= 1111 1111 1100 0111 OVERFLOW BEHAVIOUR 16 bit signed two's complement RANGE : -32768 to +32767
POSITIVE OVERFLOW: 0111 1111 1111 1111 (+32767) +0000 0000 0000 0001 (+1) 1000 0000 0000 0000
mathematical result: 32767+1 = 32768 RESULT: 1000 0000 0000 0000 = -32768 NEGATIVE OVERFLOW: 1000
0000 0000 0000 (-32768) +1111 1111 1111 1111 (-1) 1 0111 1111 1111 1111 0111 1111 1111 1111 = +32767 -32768 +(-1) =
-32769

1. **Q-5**

Textual MEDIUM 10 marks

Build the complete Hack ALU in HDL. Fill in the ALU function table for all 18 standard operations. Verify using the ALU test script.

YOUR ANSWER

CHIP ALU{ IN x[16],y[16], zx,nx, zy,ny, f,no; OUT out[16], zr, ng; PARTS: MUX16(a=x,b=false,sel=zx,out=xZero); NOT16(in=xZero,out=notX); Mux16(a=xZero,b=notX,sel=nx,out=xFinal); Mux16(a=y,b=false,sel=zy,out=yZero); Not16(in=yZero,out=NotY); Mux16(a=yZero,b=notY,sel=ny,out=yFinal); Add16(a=xFinal,b=yFinal,out=xPlusY); And16(a=xFinal,b=yFinal,out=xAndY); Mux16(a=xAndY,b=xPlusY,sel=f,out=result); Not16(in=result,out=notResult); Mux16(a=result,b=notResult,sel=no,out=out); Or8Way(in=out[0..7],out=lowBits); Or8Way(in=out[8..15],out=highBits); Or(a=lowBits,b=highBits,Out=anyBit); Not(in=anyBit,out=zr); And(a=out[15],b=true,out=ng); } zx nx zy ny f no Function 1 0 1 0 1 0 0 1 0 1 0 1 1 1 1 0 1 1 1 1 -1 0 0 1 1 1 1 x 1 1 0 0 0 0 y 0 0 1 1 0 0 !x 1 1 0 0 0 1 !y 0 0 1 1 0 1 -x 1 1 0 0 1 1 -y 0 1 1 1 1 1 x+1 1 1 0 1 1 1 y+1 0 0 1 1 1 0 x-1 1 1 0 0 1 0 y-1 0 0 0 0 1 0 x+y 0 1 0 0 1 1 x-y 0 0 0 1 1 1 y-x 0 0 0 0 0 0 x&y 0 1 0 1 0 1 x|y